

CONVOLUTIONAL AND RECURRENT NEURAL NETWORKS

Tailoring the structure of a network to the specific characteristics of the data from a task domain can reduce the training time of the network, and improves the accuracy of the network. Tailoring can be done in a number of ways, such as: constraining the connections between neurons in adjacent layers to subsets (rather than having fully connected layers); forcing neurons to share weights; or introducing backward connections into the network. Tailoring in these ways can be understood as building domain knowledge into the network. Another, related, perspective is it helps the network to learn by constraining the set of possible functions that it can learn, and by so doing guides the network to find a useful solution. It is not always clear how to fit a network structure to a domain, but for some domains where the data has a very regular structure (e.g., sequential data such as text, or gridlike

data such as images) there are well-known network architectures that have proved successful. This chapter will introduce two of the most popular deep learning architectures: convolutional neural networks and recurrent neural networks.

Convolutional Neural Networks

Convolution neural networks (CNNs) were designed for image recognition tasks and were originally applied to the challenge of handwritten digit recognition (Fukushima 1980; LeCun 1989). The basic design goal of CNNs was to create a network where the neurons in the early layer of the network would extract local visual features, and neurons in later layers would combine these features to form higher-order features. A local visual feature is a feature whose extent is limited to a small patch, a set of neighboring pixels, in an image. For example, when applied to the task of face recognition, the neurons in the early layers of a CNN learn to activate in response to simple local features (such as lines at a particular angle, or segments of curves), neurons deeper in the network combine these low-level features into features that represent body parts (such as eyes or noises), and the neurons in the final layers of the network combine body part activations in order to be able to identify whole faces in an image.

Using this approach, the fundamental task in image recognition is learning the feature detection functions that can robustly identify the presence, or absence, of local visual features in an image. The process of learning functions is at the core of neural networks, and is achieved by learning the appropriate set of weights for the connections in the network. CNNs learn the feature detection functions for local visual features in this way. However, a related challenge is designing the architecture of the network so that the network will identify the presence of a local visual feature in an image irrespective of where in the image it occurs. In other words, the feature detection functions must be able to work in a translation invariant manner. For example, a face recognition system should be able to recognize the shape of an eye in an image whether the eye is in the center of the image or in the top-right corner of the image. This need for translation invariance has been a primary design principle of CNNs for image processing, as Yann LeCun stated in 1989:

It seems useful to have a set of feature detectors that can detect a particular instance of a feature anywhere on the input plane. Since the precise location of a feature is not relevant to the classification, we can afford to lose some position information in the process. (LeCun 1989, p. 14)

CNNs achieve this translation invariance of local visual feature detection by using weight sharing between neurons. In an image recognition setting, the function implemented by a neuron can be understood as a visual feature detector. For example, neurons in the first hidden layer of the network will receive a set of pixel values as input and output a high activation if a particular pattern (local visual feature) is present in this set of pixels. The fact that the function implemented by a neuron is defined by the weights the neuron uses means that if two neurons use the same set of weights then they both implement the same function (feature detector). In chapter 4, we introduced the concept of a receptive field to describe the area that a neuron receives its input from. If two neurons share the same weights but have different receptive fields (i.e., each neuron inspects different areas of the input), then together the neurons act as a feature detector that activates if the feature occurs in either of the receptive fields. Consequently, it is possible to design a network with translation invariant feature detection by creating a set of neurons that share the same weights and that are organized so that: (1) each neuron inspects a different portion of the image; and (2) together the receptive fields of the neurons cover the entire image.

The scenario of searching an image in a dark room with a flashlight that has a narrow beam is sometimes used to explain how a CNN searches an image for local features.

At each moment you can point the flashlight at a region of the image and inspect that local region. In this flashlight metaphor, the area of the image illuminated by the flashlight at any moment is equivalent to the receptive field of a single neuron, and so pointing the flashlight at a location is equivalent to applying the feature detection function to that local region. If, however, you want to be sure you inspect the whole image, then you might decide to be more systematic in how you direct the flashlight. For example, you might begin by pointing the flashlight at the top-left corner of the image and inspecting that region. You then move the flashlight to the right, across the image, inspecting each new location as it becomes visible, until you reach the right side of the image. You then point the flashlight back to the left of the image, but just below where you began, and move across the image again. You repeat this process until you reach the bottom-right corner of the image. The process of sequentially searching across an image and at each location in the search applying the same function to the local (illuminated) region is the essence of convolving a function across an image. Within a CNN, this sequential search across an image is implemented using a set of neurons that share weights and whose union of receptive fields covers the entire image.

Figure 5.1 illustrates the different stages of processing that are often found in a CNN. The 6×6 matrix on the left of the figure represents the image that is the input to the

CNN. The 4×4 matrix immediately to the right of the input represents a layer of neurons that together search the entire image for the presence of a particular local feature. Each neuron in this layer is connected to a different 3×3 receptive field (area) in the image, and they all apply the same weight matrix to their inputs:

$$\begin{bmatrix} w_0 & w_1 & w_2 \\ w_3 & w_4 & w_5 \\ w_6 & w_7 & w_8 \end{bmatrix}$$

The receptive field of the neuron $[0,0]$ (top-left) in this layer is marked with the gray square covering the 3×3 area in the top-left of the input image. The dotted arrows emerging from each of the locations in this gray area represent the inputs to neuron $[0,0]$. The receptive field of the neighboring neuron $[0,1]$ is indicated by 3×3 square, outlined in bold in the input image. Notice that the receptive fields of these two neurons overlap. The amount of overlap of receptive fields is controlled by a hyperparameter called the stride length. In this instance, the stride length is one, meaning that for each position moved in the layer the receptive field of the neuron is translated by the same amount on the input. If the stride length hyperparameter is increased, the amount of overlap between receptive fields is decreased.

The receptive fields of both of these neurons ($[0,0]$ and $[0,1]$) are matrices of pixel values and the weights used by these neurons are also matrices. In computer vision, the matrix of weights applied to an input is known as the kernel (or convolution mask); the operation of sequentially passing a kernel across an image and within each local region, weighting each input and adding the result to its local neighbors, is known as a convolution. Notice that a convolution operation does not include a nonlinear activation function (this is applied at a later stage in processing). The kernel defines the feature detection function that all the neurons in the convolution implement. Convolving a kernel across an image is equivalent to passing a local visual feature detector across the image and recording all the locations in the image where the visual feature was present. The output from this process is a map of all the locations in the image where the relevant visual feature occurred. For this reason, the output of a convolution process is sometimes known as a feature map. As noted above, the convolution operation does not include a nonlinear activation function (it only involves a weighted summation of the inputs). Consequently, it is standard to apply a nonlinearity operation to a feature map. Frequently, this is done by applying a rectified linear function to each position in a feature map; the rectified linear activation function is defined as: $\text{rectifier}(z) = \max(0, z)$. Passing a rectified linear activation function over a feature map simply changes all

negative values to 0. In figure 5.1, the process of updating a feature map by applying a rectified linear activation function to each of its elements is represented by the layer labeled Nonlinearity.

The quote from Yann LeCun, at the start of this section, mentions that the precise location of a feature in an image may not be relevant to an image processing task. With this in mind, CNNs often discard location information in favor of generalizing the network's ability to do image classification. Typically, this is achieved by down-sampling the updated feature map using a pooling layer. In some ways pooling is similar to the convolution operation described above, in so far as pooling involves repeatedly applying the same function across an input space. For pooling, the input space is frequently a feature map whose elements have been updated using a rectified linear function. Furthermore, each pooling operation has a receptive field on the input space—although, for pooling, the receptive fields sometimes do not overlap. There are a number of different pooling functions used; the most common is called max pooling, which returns the maximum value of any of its inputs. Calculating the average value of the inputs is also used as a pooling function.

The operation sequence of applying a convolution, followed by a nonlinearity, to the feature map, and then down-sampling using pooling, is relatively standard across most CNNs. Often these three operations are together

Convolving a kernel across an image is equivalent to passing a local visual feature detector across the image and recording all the locations in the image where the visual feature was present.

considered to define a convolutional layer in a network, and this is how they are presented in figure 5.1.

The fact that a convolution searches an entire image means that if the visual feature (pixel pattern) that the function (defined by shared kernel) detects occurs anywhere in the image, its presence will be recorded in the feature map (and if pooling is used, also in the subsequent output from the pooling layer). In this way, a CNN supports translation invariant visual feature detection. However, this has the limitation that the convolution can only identify a single type of feature. CNNs generalize beyond one feature by training multiple convolutional layers in parallel (or filters), with each filter learning a single

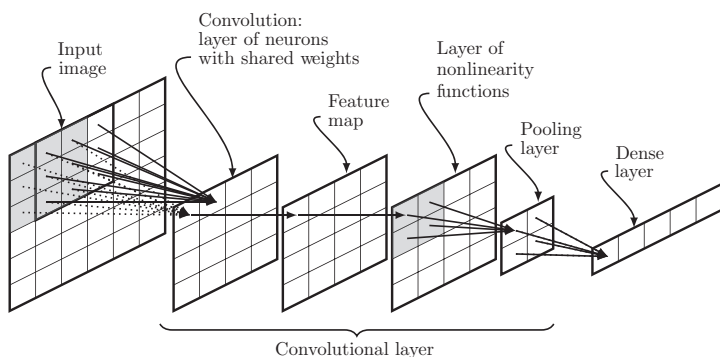


Figure 5.1 Illustrations of the different stages of processing in a convolutional layer. Note in this figure the Image and Feature Map are data structures; the other stages represent operations on data.

kernel matrix (feature detection function). Note the convolution layer in figure 5.1 illustrates a single filter. The outputs of multiple filters can be integrated in a variety of ways. One way to integrate information from different filters is to take the feature maps generated by the separate filters and combine them into a single multifilter feature map. A subsequent convolutional layer then takes this multifilter feature map as input. Another other way to integrate information from different filter is to use a densely connected layer of neurons. The final layer in figure 5.1 illustrates a dense layer. This dense layer operates in exactly the same way as a standard layer in a fully connected feedforward network. Each neuron in the dense layer is connected to all of the elements output by each of the filters, and each neuron learns a set of weights unique to itself that it applies to the inputs. This means that each neuron in a dense layer can learn a different way to integrate information from across the different filters.

The AlexNet CNN, which won the ImageNet Large-Scale Visual Recognition Challenge (ILSVRC) in 2012, had five convolutional layers, followed by three dense layers. The first convolutional layer had ninety-six different kernels (or filters) and included a ReLU nonlinearity and pooling. The second convolution layer had 256 kernels and also included ReLU nonlinearity and pooling. The third,

fourth, and fifth convolutional layers did not include a nonlinearity step or pooling, and had 384, 384, and 256 kernels, respectively. Following the fifth convolutional layer, the network had three dense layers with 4096 neurons each. In total, AlexNet had sixty million weights and 650,000 neurons. Although sixty million weights is a large number, the fact that many of the neurons shared weights actually reduced the number of weights in the network. This reduction in the number of required weights is one of the advantages of CNN networks. In 2015, Microsoft Research developed a CNN network called ResNet, which won the ILSVRC 2015 challenge (He et al. 2016). The ResNet architecture extended the standard CNN architecture using skip-connections. A skip-connection takes the output from one layer in the network and feeds it directly into a layer that may be much deeper in the network. Using skip-connections it is possible to train very deep networks. In fact, the ResNet model developed by Microsoft Research had a depth of 152 layers.

Recurrent Neural Networks

Recurrent neural networks (RNNs) are tailored to the processing of sequential data. An RNN processes a sequence of data by processing each element in the sequence one at a time. An RNN network only has a single hidden

layer, but it also has a memory buffer that stores the output of this hidden layer for one input and feeds it back into the hidden layer along with the next input from the sequence. This recurrent flow of information means that the network processes each input within the context generated by processing the previous input, which in turn was processed in the context of the input preceding it. In this way, the information that flows through the recurrent loop encodes contextual information from (potentially) all of the preceding inputs in the sequence. This allows the network to maintain a memory of what it has seen previously in the sequence to help it decide what to do with the current input. The depth of an RNN arises from the fact that the memory vector is propagated forward and evolved through each input in the sequence; as a result an RNN network is considered as deep as a sequence is long.

Figure 5.2 illustrates the architecture of an RNN and shows how information flows through the network as it processes a sequence. At each time step, the network in this figure receives a vector containing two elements as input. The schematic on the left of figure 5.2 (time step=1.0) shows the flow of information in the network when it receives the first input in the sequence. This input vector is fed forward into the three neurons in the hidden layer of the network. At the same time these neurons also receive whatever information is stored in the memory

The depth of an RNN arises from the fact that the memory vector is propagated forward and evolved through each input in the sequence; as a result an RNN network is considered as deep as a sequence is long.

buffer. Because this is the initial input, the memory buffer will only contain default initialization values. Each of the neurons in the hidden layer will process the input and generate an activation. The schematic in the middle of figure 5.2 (time step=1.5) shows how this activation flows on through the network: the activation of each neuron is passed to the output layer where it is processed to generate the output of the network, and it is also stored in the memory buffer (overwriting whatever information was stored there). The elements of the memory buffer simply store the information written to them; they do not transform it in any way. As a result, there are no weights on the edges going from the hidden units to the buffer. There are, however, weights on all the other edges in the network, including those from the memory buffer units to the neurons in the hidden layer. At time step 2, the network receives the next input from the sequence, and this is passed to the hidden layer neurons along with the information stored in the buffer. This time the buffer contains the activations that were generated by the hidden neurons in response to the first input.

Figure 5.3 shows an RNN that has been unrolled through time as it processes a sequence of inputs $[X_1, X_2, \dots, X_t]$. Each box in this figure represents a layer of neurons. The box labeled h_0 represents the state of the memory buffer when the network is initialized; the boxes labeled $[h_1, \dots, h_t]$ represent the hidden layer of the

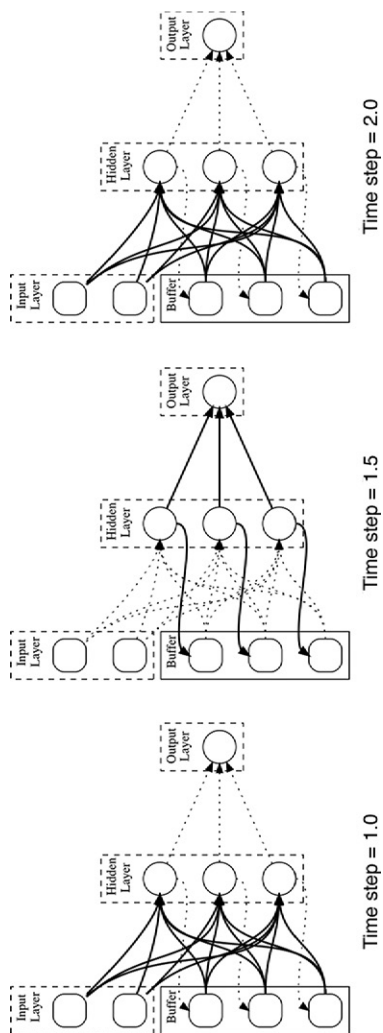


Figure 5.2 The flow of information in an RNN as it processes a sequence of inputs. The arrows in bold are the active paths of information flow at each time point; the dashed arrows show connections that are not active at that time.

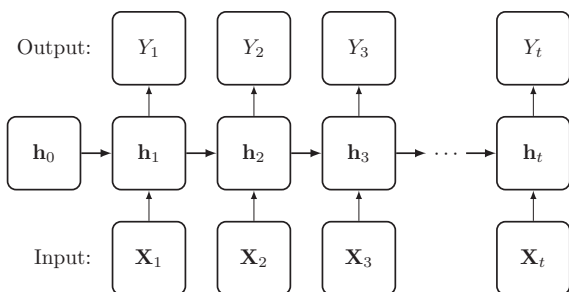


Figure 5.3 An RNN network unrolled through time as it processes a sequence of inputs $[X_1, X_2, \dots, X_t]$.

network at each time step; and the boxes labeled $[Y_1, \dots, Y_t]$ represent the output layer of the network at each time step. Each of the arrows in the figure represents a set of connections between one layer and another layer. For example, the vertical arrow from X_1 to h_1 represents the connections between the input layer and the hidden layer at time step 1. Similarly, the horizontal arrows connecting the hidden layers represent the storing of the activations from a hidden state at one time step in the memory buffer (not shown) and the propagation of these activations to the hidden layer at the next time step through the connections from the memory buffer to the hidden state. At each time step, an input from the sequence is presented to the network and is fed forward to the hidden layer. The hidden layer generates a vector of activations that is passed to the output layer and is also propagated forward to the

next time step along the horizontal arrows connecting the hidden states.

Although RNNs can process a sequence of inputs, they struggle with the problem of vanishing gradients. This is because training an RNN to process a sequence of inputs requires the error to be backpropagated through the entire length of the sequence. For example, for the network in figure 5.3, the error calculated on the output Y_t must be backpropagated through the entire network so that it can be used to update the weights on the connections from h_0 and X_1 to h_1 . This entails backpropagating the error through all the hidden layers, which in turn involves repeatedly multiplying the error by the weights on the connections feeding activations from one hidden layer forward to the next hidden layer. A particular problem with this process is that it is the same set of weights that are used on all the connections between the hidden layers: each horizontal arrow represents the same set of connections between the memory buffer and the hidden layer, and the weights on these connections are stationary through time (i.e., they don't change from one time step to the next during the processing of a given sequence of inputs). Consequently, backpropagating an error through k time steps involves (among other multiplications) multiplying the error gradient by the same set of weights k times. This is equivalent to multiplying each error gradient by a weight raised to the power of k . If this weight is

less than 1, then when it is raised to a power, it diminishes at an exponential rate, and consequently, the error gradient also tends to diminish at an exponential rate with respect to the length of the sequence—and vanish.

Long short-term memory networks (LSTMs) are designed to reduce the effect of vanishing gradients by removing the repeated multiplication by the same weight vector during backpropagation in an RNN. At the core of an LSTM¹ unit is a component called the cell. The cell is where the activation (the short-term memory) is stored and propagated forward. In fact, the cell often maintains a vector of activations. The propagation of the activations within the cell through time is controlled by three components called gates: the forget gate, the input gate, and the output gate. The forget gate is responsible for determining which activations in the cell should be forgotten at each time step, the input gate controls how the activations in the cell should be updated in response to the new input, and the output gate controls what activations should be used to generate the output in response to the current input. Each of the gates consists of layers of standard neurons, with one neuron in the layer per activation in the cell state.

Figure 5.4 illustrates the internal structure of an LSTM cell. Each of the arrows in this image represents a vector of activations. The cell runs along the top of the figure from left (c_{t-1}) to right (c_t). Activations in the cell

can take values in the range -1 to +1. Stepping through the processing for a single input, the input vector x_t is first concatenated with the hidden state vector that has been propagated forward from the preceding time step h_{t-1} . Working from left to right through the processing of the gates, the forget gate takes the concatenation of the input and the hidden state and passes this vector through a layer of neurons that use a sigmoid (also known as logistic)² activation function. As a result of the neurons in the forget layer using sigmoid activation functions the output of this forget layer is a vector of values in the range 0 to 1. The cell state is then multiplied by this forget vector. The result of this multiplication is that activations in the cell state that are multiplied by components in the forget vector with values near 0 are forgotten, and activations that are multiplied by forget vector components with values near 1 are remembered. In effect, multiplying the cell state by the output of a sigmoid layer acts as a filter on the cell state.

Next, the input gate decides what information should be added to the cell state. The processing in this step is done by the components in the middle block of figure 5.4, marked Input. This processing is broken down into two subparts. First, the gate decides which elements in the cell state should be updated, and second it decides what information should be included in the update. The decision regarding which elements in the cell state should be updated is implemented using a similar filter mechanism

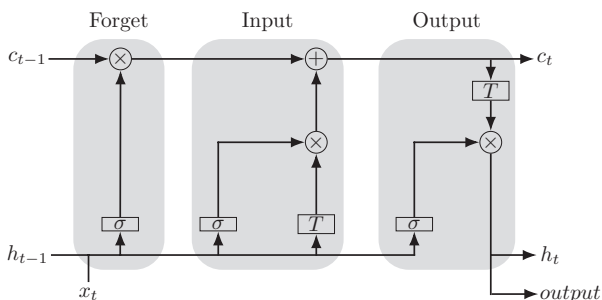


Figure 5.4 Schematic of the internal structure of an LSTM unit: σ represents a layer of neurons with sigmoid activations, T represents a layer of neurons with tanh activations, $\otimes$ represents vector multiplication, and $+$ represents vector addition. The figure is inspired by an image by Christopher Olah available at: <http://colah.github.io/posts/2015-08-Understanding-LSTMs/>.

to the forget gate: the concatenated input x_t plus hidden state h_{t-1} is passed through a layer of sigmoid units to generate a vector of elements, the same width as the cell, where each element in the vector is in the range 0 to 1; values near 0 indicate that the corresponding cell element will not be updated, and values near 1 indicate that the corresponding cell element will be updated. At the same time that the filter vector is generated, the concatenated input and hidden state are also passed through a layer of tanh units (i.e., neurons that use the tanh activation function). Again, there is one tanh unit for each activation in the LSTM cell. This vector represents the information

that may be added to the cell state. Tanh units are used to generate this update vector because tanh units output values in the range -1 to +1, and consequently the value of the activations in the cell elements can be both increased and decreased by an update.³ Once these two vectors have been generated, the final update vector is calculated by multiplying the vector output from the tanh layer by the filter vector generated from the sigmoid layer. The resulting vector is then added to the cell using vector addition.

The final stage of processing in an LSTM is to decide which elements of the cell should be output in response to the current input. This processing is done by the components in the block marked Output (on the right of figure 5.4). A candidate output vector is generated by passing the cell through a tanh layer. At the same time, the concatenated input and propagated hidden state vector are passed through a layer of sigmoid units to create another filter vector. The actual output vector is then calculated by multiplying the candidate output vector by this filter vector. The resulting vector is then passed to the output layer, and is also propagated forward to the next time step as the new hidden state h_t .

The fact that an LSTM unit contains multiple layers of neurons means that an LSTM is a network in itself. However, an RNN can be constructed by treating an LSTM as the hidden layer in the RNN. In this configuration, an

LSTM unit receives an input at each time step and generates an output for each input. RNNs that use LSTM units are often known as LSTM networks.

LSTM networks are ideally suited for natural language processing (NLP). A key challenge in using a neural network to do natural language processing is that the words in language must be converted into vectors of numbers. The word2vec models, created by Tomas Mikolov and colleagues at Google research, are one of the most popular ways of doing this conversion (Mikolov et al. 2013). The word2vec models are based on the idea that words that appear in similar contexts have similar meanings. The definition of context here is surrounding words. So for example, the words *London* and *Paris* are semantically similar because each of them often co-occur with words that the other word also co-occurs with, such as: *capital*, *city*, *Europe*, *holiday*, *airport*, and so on. The word2vec models are neural networks that implement this idea of semantic similarity by initially assigning random vectors to each word and then using co-occurrences within a corpus to iteratively update these vectors so that semantically similar words end up with similar vectors. These vectors (known as word embeddings) are then used to represent a word when it is being input to a neural network.

One of the areas of NLP where deep learning has had a major impact is in machine translation. Figure 5.5 presents a high-level schematic of the seq2seq (or

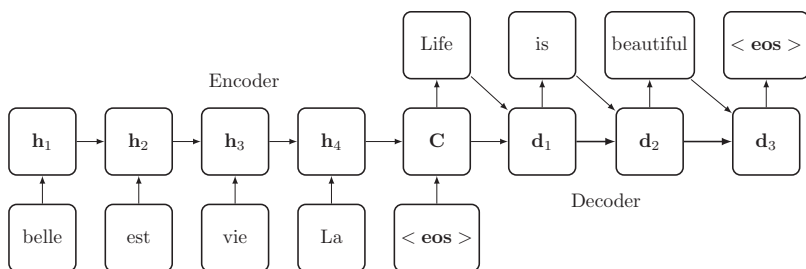


Figure 5.5 Schematic of the seq2seq (or encoder-decoder) architecture.

encoder-decoder) architecture for neural machine translation (Sutskever et al. 2014). This architecture is composed of two LSTM networks that have been joined together. The first LSTM network processes the input sentence in a word-by-word fashion. In this example, the source language is French. The words are entered into the system in reverse order as it has been found that this leads to better translations. The symbol eos is a special end of sentence symbol. As each word is entered, the encoder updates the hidden state and propagates it forward to the next time step. The hidden state generated by the encoder in response to the eos symbol is taken to be a vector representation of the input sentence. This vector is passed as the initial input to the decoder LSTM. The decoder is trained to output the translation sentence word by word, and after each word has been generated, this word is fed back into the system as the input for the next time step. In a

way, the decoder is hallucinating the translation because it uses its own output to drive its own generation process. This process continues until the decoder outputs an eos symbol.

The idea of using a vector of numbers to represent the (interlingual) meaning of a sentence is very powerful, and this concept has been extended to the idea of using vectors to represent intermodal/multimodal representations. For example, an exciting development in recent years has been the development of automatic image captioning systems. These systems can take an image as input and generate a natural language description of the image. The basic structure of these systems is very similar to the neural machine translation architecture shown in figure 5.5. The main difference is that the encoder LSTM network is replaced by a CNN architecture that processes the input image and generates a vector representation that is then propagated to the decoder LSTM (Xu et al. 2015). This is another example of the power of deep learning arising from its ability to learn complex representations of information. In this instance, the system learns intermodal representations that enable information to flow from what is in an image to language. Combining CNN and RNN architectures is becoming more and more popular because it offers the potential to integrate the advantages of both systems and enables deep learning architectures to handle very complex data.

Irrespective of the network architecture we use, we need to find the correct weights for the network if we wish to create an accurate model. The weights of a neuron determine the transformation the neuron applies to its inputs. So, it is the weights of the network that define the fundamental building blocks of the representation the network learns. Today the standard method for finding these weights is an algorithm that came to prominence in the 1980s: backpropagation. The next chapter will present a comprehensive introduction to this algorithm.